通用多智能体 Auto-Task 架构设计

通用多智能体 Auto-Task 架构设计

> 兼顾效率、可扩展性、通用性，统一支持流式（Streaming）与定时（Cron）触发。 以“定时邮件 / 定期推送”为贯穿示例。

目录

1. 设计目标与核心抽象
2. 整体架构图
3. 模块详细分析
   - 3.1 Trigger Layer
   - 3.2 Event Bus
   - 3.3 Router / Dispatcher
   - 3.4 Agent Runtime
   - 3.5 Memory Layer
   - 3.6 Tool Gateway
   - 3.7 Model Router
   - 3.8 Cross-Cutting
4. 端到端案例：定时邮件回复 / 定期推送
5. 关键设计决策与 Trade-offs
6. 演进路径
7. 总结

---

1. 设计目标与核心抽象

1.1 设计目标

目标	含义	落地手段
通用性	支持任意 task / 任意触发源	声明式 manifest + 可插拔 Emitter
效率	高吞吐 + 低延迟 + 低成本	微批 + 分区 + 池化 + 模型分级
可扩展性	新事件 / 新 agent / 新工具零侵入	事件总线解耦 + Schema Registry
流式 + 定时统一	两种触发同源同流处理	Cron 视为周期性事件源
多智能体协作	Agent 之间天然组合	通过事件总线通信，不走 RPC

1.2 核心抽象（三个一等公民）

概念	职责	例子
Trigger	决定“何时”	cron / stream / webhook / manual / agent-emit
Event	统一的触发信号（不可变 envelope）	`email.received`，`cron.tick.daily`
Agent	决定“做什么”（LLM + Memory + Tools + Goal）	`email-triage`，`email-draft`

关键设计原则：Trigger 与 Agent 通过 Event 完全解耦。 这一刀切下去，“流式 + 定时 + 任意未来触发源 + 任意未来 agent” 就天然可组合。

---

2. 整体架构图

```
╔═══════════════════════════════════════════════════════════════════════════════════════════════════════════
│                          EXTERNAL  WORLD                                                    │
│   CLI / UI    Webhooks    Files / DB CDC    Schedules    上游 Agent 输出          │
╚═══════╤═══════╤═══════╤═══════╤═══════╤═══════╤═══════╤═══════╤═══════╤═══════╤═══════╤═══
    │           │               │                  │             │
────┼───────────┼───────────────┼──────────────────┼─────────────┼──────────────────────────────┼────
    ▼           ▼               ▼                  ▼             ▼

┌──────────────────────────────────────────────────────────────────────────────────────────────────────
│  ①  TRIGGER  LAYER   (可插拔 Emitter, 统一输出 Event Envelope)                    │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────┐
┌──────────────────────────────────────────┐ │
│  │ Manual │  │ Webhook │  │  Stream  │  │    Cron     │  │  AgentEmitter          │ │
│  │        │  │ HMAC+   │  │ 微批+    │  │ timing-wheel│  │  agent 自身输出        │ │
│  │        │  │ replay  │  │ 分区     │  │ + leader    │  │  → multi-agent 协作    │ │
│  └────────┴──┘         └──┘          └──┘             └──┘                        └──┘
└──────────────────────────────────────────┘ │
└──────────┴──────────────┴──────────────┴──────────────┴──────────────────────────────────────────────

└──────────────┴──────────────┴──────────────┴──────────────────────────────────────────────────────────

                                  │  Envelope {topic, schema_v, ts, dedup_key,
                                  │            trace_id, payload, source}
                                  ▼

┌──────────────────────────────────────────────────────────────────────────────────────────────────────
│  ②  EVENT  BUS   (Kafka / Pulsar)                                                  │
│       topic 命名: domain.entity.action     partition by dedup_key                  │
│
┌──────────────────────────────────────────────────────────────────────────────────────────────────────
│
│   │  cron.tick.*  │  email.*  │  agent.*  │  approval.*  │  github.*  ···    │      │
│
└──────────────────────────────────────────────────────────────────────────────────────────────────────
│
│       Schema Registry (envelope 只增不改 + payload schema 版本化)                  │
└──────────────────────────────────────────────────────────────────────────────────────────────────────
                                  ▼

┌──────────────────────────────────────────────────────────────────────────────────────────────────────
│  ③  ROUTER  /  DISPATCHER   (stateless, 水平扩展)                                  │
│   topic→agent 路由  │  dedup 去重  │  rate-limit  │  优先级  │  DAG / 链式触发  │
│   双轨路由: 配置规则优先匹配 → 未命中走 LLM 智能路由                                  │
└──────────────────────────────────────────────────────────────────────────────────────────────────────
                                  ▼

╔═══════════════════════════════════════════════════════════════════════════════════════════════════════
│  ④  🧠  AGENT  RUNTIME   (核心新增层)                                                 │
│                                                                                       │
│
┌──────────────────────────────────────────────────────────────────────────────────────────────────────
│
│    │   Agent Loader  ──→  按 manifest 实例化 / 从 checkpoint 恢复          │        │
│    │        │                                                              │        │
│    │        ▼                                                              │        │
│    │
┌──────────────────────────────────────────────────────────────────────────────────────────────────────
│        │
│    │    │   ┌──────────────────────┐     ┌──────────────────────┐
┌──────────────────────┐     │    │        │
│    │    │   │ Context      │  →  │ LLM        │  →  │ Tool         │     │    │        │
│    │    │   │ Builder      │     │ Inference  │     │ Dispatcher   │     │    │        │
│    │    │   │(memory+event)│     │ (think)    │     │ (act)        │     │    │        │
│    │    │   └──────────────────────┘     └──────────────────────┘
└──────────────┴──────────────────────┘     │    │        │
```

```
│    │    │          ▲                                    │              │    │        │
│    │    │          │              ◄──── observe ◄──────────────────────┘              │    │
│
│    │    │          └──────────────────────────────── loop
──────────────────────────────                 │    │        │
│    │
└──────────────────────────────
```

2.1 数据流编号

```
外部信号
    │
    ▼  ①  Emitter 把异构源转成统一 envelope
Trigger Layer
    │
    ▼  ②  发布到 Event Bus（按 dedup_key 分区）
Event Bus  ◄──────────────────────────────────────────────────────────┐
    │                                                                 │
    ▼  ③  Router 按 topic 路由 + 去重 + 限流                          │
Router                                                                │
    │                                                                 │
    ▼  ④  Agent Runtime 加载 agent 实例                               │
Agent (think-act-observe loop)                                        │
    │  │  │                                                          │
    │  │  └──────→ ⑤ 读写 Memory (Working/Episodic/Sem)              │
    │  └──────────→ ⑥ 调用 Tool (经 Gateway 鉴权/审计)               │
    └──────────────→ ⑦ 推理 (Model Router 选模型/走 Cache)           │
    │                                                                 │
    ▼  agent 输出可发新事件                                           │
 emit_event ──────────────────────────────────────────────────────────►┘
                (multi-agent 协作天然形成)
```

2.2 流式 + 定时融合的关键

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│        │         Long-Running Agent  内部循环              │
│        │                                                   │
│        │    ┌──────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│
│        │    │    wait_any(  inbox  ,  next_wakeup ) │       │
│        │    └──────────────┬──────────────────────────────────────────────────────────┬──────────────────────────────────┘
│
│        │           │                     │                │
│        │     [流式事件到达]        [定时器到点]            │
│        │           │                     │                │
│        │           └─────────────────────┴────────────────┘                 │
│        │                     ▼                            │
│        │               ┌──────────────────────────┐                        │
│        │               │  LLM think     │                  │
│        │               │ + tool act     │                  │
│        │               │ + memory r/w   │                  │
│        │               └──────────────┬───────────────┘                        │
│        │                     ▼                            │
│        │            ┌──────────────────────────────────────┐                 │
│        │            │ LLM 自决:            │                │
│        │            │ next_wakeup = ?      │ ←─── 动态自适应│
│        │            │ (5min / 1h / 不再醒) │                │
│        │            └──────────────────────┴────────────────┘                 │
│        │                     │                            │
│        │                     └──────── 回到 wait_any ────────────────┤
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

> 流式给 inbox，定时给 timer，Agent 自己决定下一次什么时候醒——这是流式与定时的真正统一。

3. 模块详细分析

3.1 Trigger Layer（触发层）

职责：把异构外部信号（HTTP / 文件 / cron / MQ / agent 输出）转成统一 Event Envelope，发布到总线。这一层是兼容性的入口。

内部组成

Emitter	实现要点
`ManualEmitter`	CLI / UI 直接 publish，主要用于开发调试
`WebhookEmitter`	HTTP server + HMAC 签名校验 + nonce 去重 + 5 分钟时间窗
`StreamEmitter`	长连接（SSE / WS / Kafka 消费），100ms 微批聚合，按 dedup_key 分区
`CronEmitter`	timing wheel（O(1) 入队）+ Redis/etcd leader election 防多实例重复
`AgentEmitter`	Agent 输出直接 publish，让 multi-agent 协作走总线

关键设计

- Emitter 接口统一：`emit(envelope) → bus`，新源 = 新实现，平台代码零改动
- Cron 高可用：多实例 + 分布式锁选主；持久化 `last_fire_ts`，重启后 catch-up 漏触发
- 背压：StreamEmitter 监控 bus producer lag，超阈值触发 token bucket 限流
- 重放保护：WebhookEmitter 用 nonce + 时间窗，5 分钟内同 nonce 拒绝

Envelope 格式

```
{
  "event_id":       "uuid",
  "topic":          "domain.entity.action",
  "schema_version": "v1",
  "ts":             1714521600,
  "source":         "stream-emitter-7",
  "dedup_key":      "msg:abc123",
  "trace_id":       "...",
  "tenant_id":      "...",
  "payload":        { ... }
}
```

Trade-off

- 微批 vs 实时：100ms 聚合损失少量延迟，换 10× 吞吐——绝大多数场景值
- Cron 集中 vs 分散：集中（一个 CronEmitter 服务）易管理；分散（每个 agent 自带 cron）灵活但难限流。选集中。

3.2 Event Bus（事件总线）

职责：所有事件的统一中枢，提供持久化、有序、可重放的消息传递。

选型对比

方案	优势	劣势	适用
Kafka	吞吐巨大、生态完善	无原生 delay queue、运维重	大规模生产
Pulsar	多租户、原生 delay、分层存储	社区小一些	多租户平台
NATS JetStream	轻量、低延迟	吞吐不及 Kafka	中小规模
Redis Streams	部署简单	持久化弱、规模有限	早期 / 单机

默认选 Kafka，delay 场景外挂 Pulsar 或自建 delay topic。

关键设计

- Topic 命名：`domain.entity.action`（如 `email.received`），支持通配订阅 `email.*`
- 分区策略：`partition = hash(dedup_key) % N`，保证同 key 顺序 + worker 水平扩展
- Schema Registry：每个 topic 注册 payload schema + 版本，只增字段不改语义
- 保留期分级：高频事件 1 天，业务事件 7 天，审计事件 30 天
- 投递语义：at-least-once，业务侧用 `dedup_key` 幂等

关键决策：要不要 exactly-once？

不要。代价 = 严重吞吐损失 + 复杂度。业务幂等才是正解。

Trade-off

- 分区数：多了 → 并发高但 rebalance 慢；少了 → 吞吐瓶颈。经验值：预估峰值 QPS / 单分区 5k
- 保留期：长 → 可重放但贵；短 → 省钱但丢历史。按事件价值分级

3.3 Router / Dispatcher（路由分发层）

职责：把事件路由到正确的 agent，附带去重、限流、优先级、链式编排。stateless，水平扩展。

内部组成

```
┌──────────────────────────────────────────────────────────────────────────────────────┐
│  Topic Matcher (规则 + 通配)              │
│       ↓                                    │
│  Idempotency Filter (查 Redis)            │
│       ↓                                    │
│  Rate Limiter (token bucket per agent)    │
│       ↓                                    │
│  Priority Queue (high/normal/low)         │
│       ↓                                    │
│  Agent Dispatcher → Agent Runtime          │
└──────────────────────────────────────────────────────────────────────────────────────┘
```

关键设计

- 双轨路由
  - Fast path（配置规则）：`topic_pattern → agent_id` YAML，毫秒级匹配
  - Slow path（LLM 智能路由）：未命中规则 → 调用 Haiku 判断该交给哪个 agent
- 去重：`SETNX dedup_key EX ttl`，已存在则丢弃
- 限流：每个 agent 一个 token bucket（QPS / concurrency 双约束）
- DAG / 链式：agent A 完成发新事件 → router 触发 agent B（不在 router 里硬编码 DAG）

- 环检测：trace_id 串起整条链，深度超阈值 / 检出环则告警 + 截断

路由表示例

```yaml
routes:
  - topic: email.received
    agent: email-triage
    priority: high
  - topic: email.draft.requested
    agent: email-draft
    rate_limit: { qps: 5 }
  - topic: cron.tick.email-followup
    agent: email-followup
  - topic: email.draft.ready
    agent: email-send
  - topic: github.*               # 通配
    agent: github-handler
```

3.4 Agent Runtime（智能体运行时——核心）

职责：执行 agent 的 think-act-observe 循环，管理生命周期、状态、预算、安全。

内部组成

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│   Agent Loader                                                                                                           │
│   ─ 按 manifest 实例化 / 从 checkpoint 恢复                                                                                 │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│   Context Builder                                                                                                        │
│   ─ 拉 working / episodic / semantic memory                                                                              │
│   ─ 拼 system prompt + recent events                                                                                     │
│   ─ token budget 控制（超了走 compression）                                                                                 │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
          ┌──────────────────────────────────────────┐
          │   ┌──────────────────────────────┐       │
          │   │ LLM Inference │   │   ← 通过 Model Router
          │   │   (think)     │   │
          │   └──────────────┬───────────────┘       │
          │                  ▼                       │
          │   ┌──────────────────────────────┐       │
          │   │ Tool Dispatch │   │   ← 通过 Tool Gateway
          │   │   (act)       │   │
          │   └──────────────┬───────────────┘       │
          │                  ▼                       │
          │   ┌──────────────────────────────┐       │
          │   │  Observation  │   │
          │   │  (write back  │   │
          │   │   to memory)  │   │
          │   └──────────────┬───────────────┘       │
          │                  │                       │
          │   ◄──────────────┘                       │
          │   loop until done   │
          └──────────────────────────────────────────┘
                                  ▼
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│   横切控制                                                                                                                 │
│   • Budget Guard   (tokens / cost / walltime)                                                                            │
│   • Checkpoint     (每 N turns 持久化状态)                                                                                  │
│   • Approval Hook  (高危工具调用拦截)                                                                                        │
│   • Output Schema  (LLM 输出 JSON Schema 校验)                                                                              │
│   • Emit Event     (输出 follow-up event)                                                                                  │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

三种 Agent 形态

形态	进程模型	状态	适用
Reactive	短进程，pool 复用	无状态	分类、回复
Periodic	短进程，每次重建	通过 memory 持久化	监控、巡检
Long-running	常驻 actor，状态在内存	内存 + 周期性 checkpoint	协作、复杂状态机

Long-running Agent 核心循环（伪代码）

```python
async def run(self):
    while not self.done:
        signal = await wait_any(self.inbox, timer(self.next_wakeup))
        plan   = await self.llm.think(self.memory, signal)
        result = await self.tools.execute(plan.actions)
        self.memory.update(result)
        # LLM 自决下一次唤醒时间（动态自适应）
        self.next_wakeup = self.llm.decide_wakeup(result)
        if self.turn_count % CHECKPOINT_EVERY == 0:
            await self.persist()
```

关键设计

- 状态序列化：memory + inbox cursor + turn_count → JSON 落 PG，崩溃可恢复
- Budget Guard：每次 LLM 调用前预估 tokens，超 budget 直接终止 + emit `agent.budget.exceeded`
- Approval Hook：tool manifest 标记 `require_approval: true` → runtime 暂停 + 发审批 → 等回调
- Output Schema 校验：LLM 返回 JSON 不符 schema → 自动 retry（最多 N 次）→ 还不行 emit 错误事件

Trade-off

- 长寿命 vs 短寿命：长寿命延迟低、context warm，但占内存；短寿命可弹性伸缩。默认 reactive
- Sub-agent 派生 vs 单 agent 长上下文：派生隔离 context 但增加协调；长上下文简单但易爆。长流程必派生

3.5 Memory Layer（记忆层）

职责：给 agent 提供短期/中期/长期记忆，支撑跨 run 的状态延续和经验复用。

三层结构

层	存储	内容	写入时机	读取时机
Working	Redis KV	当前 run 的变量/计数器/中间状态	tool 执行后	每个 turn 开始
Episodic	Postgres	历史 event/action/observation 序列	每个 turn 结束	run 启动 / 回放
Semantic	Vector DB (Weaviate/Qdrant)	知识、文档、经验摘要	显式 `remember()`	retrieval-on-demand

Context Builder 算法

```
budget = max_tokens - reserved_for_response
1. 拉 system prompt                    [固定]
2. 拉 working memory 全量              [必含]
3. 拉最近 N 条 episodic（按时间倒序）    [可裁剪]
4. 按当前 query 向 semantic 检索 top-K  [可裁剪]
5. 拼装：system + semantic + episodic + working + current_event
6. 若超 budget → 触发 compression（旧 episodic 摘要替换）
```

关键设计

- Compression 策略：滑动窗口 + LLM 摘要旧 turns，把详细 trace 换成短摘要
- Sub-agent 隔离：父 agent 派生子 agent 时，子 agent context 全新，结果摘要回流

- Semantic write 节流：不是每个 observation 都写 vector DB，要 LLM 判断“值不值得记”
- Memory namespace：`agent_id : tenant_id : session_id`，多租户硬隔离

Trade-off

- 全量上下文 vs 摘要：全量准确但贵；摘要省钱但丢细节。冷数据摘要、热数据全量
- 共享 memory vs 隔离：共享便于协作但隔离性差。默认隔离，shared 显式声明

3.6 Tool Gateway（工具网关）

职责：所有工具调用的统一入口，提供注册、鉴权、沙箱、审计。

内部组成

```
┌──────────────────────────────────────────────────────────────────────┐
│  Tool Registry (MCP-style)          │
│  ├─ 工具元信息 / schema             │
│  ├─ 版本 / 兼容性                   │
└──────────────────────┬──────────────────────────────────────────────┘
          ▼
┌──────────────────────────────────────────────────────────────────────┐
│  Permission Check                   │
│  ├─ agent → tool ACL                │
│  ├─ tenant → tool quota             │
└──────────────────────┬──────────────────────────────────────────────┘
          ▼
┌──────────────────────────────────────────────────────────────────────┐
│  Approval Hook (高危工具)           │
│  ├─ 暂停 → 通知 → 等审批回调        │
└──────────────────────┬──────────────────────────────────────────────┘
          ▼
┌──────────────────────────────────────────────────────────────────────┐
│  Sandbox Executor                   │
│  ├─ file: chroot / 容器             │
│  ├─ network: egress allowlist       │
│  ├─ exec: gVisor / firecracker      │
└──────────────────────┬──────────────────────────────────────────────┘
          ▼
┌──────────────────────────────────────────────────────────────────────┐
│  Audit Log (every call)             │
└──────────────────────────────────────────────────────────────────────┘
```

关键设计

- MCP 协议统一：所有工具实现 `{ name, schema, invoke(input) → output }`，新工具即插即用
- 三级权限沙箱
  - L1（read-only）：本地直接跑
  - L2（写本地 / 调外部 API）：容器隔离
  - L3（执行任意代码 / 高权限 API）：microVM + approval
- Approval 流程：tool manifest 标记 → runtime 调用前发审批请求到 Slack/钉钉 → 用户回调 → 继续/拒绝
- Audit log：每次调用持久化 `{agent, tool, input, output, ts, trace_id}`，可回溯/合规
- Idempotency：工具调用支持 `idempotency_key`，重复调用返回原结果

Trade-off

- 沙箱强度 vs 启动延迟：microVM 强但启动慢；容器弱但快。按权限级别选
- 集中 Gateway vs 直连：集中便于审计但加一跳延迟；直连快但失控。业务工具集中，热路径白名单直连

3.7 Model Router（模型路由层）

职责：根据任务复杂度、预算、延迟要求，把推理请求路由到合适的模型，最大化 cost / quality 比。

内部组成

```
┌──────────────────────────────────────────────────────────────┐
│  Request Classifier                  │
│  ─ Haiku 快速判断任务复杂度          │
│  ─ 或按 manifest 静态指定            │
└──────────────────────┬───────────────────────────────────────┘
          ▼
┌──────────────────────────────────────────────────────────────┐
│  Model Selector                      │
│  ─ Haiku   (路由/分类/简单回复)      │
│  ─ Sonnet  (默认, 性价比高)          │
│  ─ Opus    (复杂推理/规划)           │
└──────────────────────┬───────────────────────────────────────┘
          ▼
┌──────────────────────────────────────────────────────────────┐
│  Prompt Cache Manager                │
│  ─ system prompt + 长 context 缓存 │
│  ─ 命中率监控                        │
└──────────────────────┬───────────────────────────────────────┘
          ▼
┌──────────────────────────────────────────────────────────────┐
│  Provider Adapter                    │
│  ─ Anthropic / OpenAI / 本地         │
│  ─ 故障切换 / 重试                   │
└──────────────────────┬───────────────────────────────────────┘
          ▼
┌──────────────────────────────────────────────────────────────┐
│  Cost Tracker                        │
│  ─ per-agent / per-tenant 计费       │
└──────────────────────────────────────────────────────────────┘
```

关键设计

- Adaptive 降级：tenant 当月预算剩 10% → 自动 Opus → Sonnet 降级 + 通知
- Prompt Cache 优化：固定 system prompt 写最前，提升缓存命中（Anthropic prompt cache TTL 5 分钟）
- Streaming 支持：长输出走 streaming，降低首 token 延迟
- Provider 抽象：统一适配 Anthropic / OpenAI / 自部署，故障可切换

Trade-off

- 智能路由 vs 静态指定：智能贵但精准；静态便宜但僵硬。默认静态，复杂任务智能
- 多 provider vs 单 provider：多 provider 容灾但适配成本；单 provider 简单但有锁定风险。关键场景多 provider

---

3.8 Cross-Cutting（横切关注点）

不是单独服务，而是贯穿所有模块的能力。

3.8.1 State Store

- Postgres：持久化 agent state、checkpoint、event 审计、approval 记录
- Redis：幂等去重（SETNX）、限流计数、分布式锁、agent inbox
- 设计原则：热路径 Redis（毫秒级），持久化 Postgres（强一致）

3.8.2 DLQ + Replay

- 路由 / 执行失败的事件进 DLQ topic
- Replay 工具：CLI / UI 触发重放，可改路由 / payload 后重发
- 保护机制：DLQ 也有上限，溢出告警

3.8.3 Observability

不只是普通指标，还要 LLM trace：

- Metrics（Prometheus）：QPS / 延迟 / 错误率 / 队列深度 / agent concurrency
- Trace（OpenTelemetry）：跨 emitter → bus → router → agent → tool 全链路 trace_id
- LLM Trace（Langfuse）：每次推理的 prompt / completion / tokens / cost / model / latency
- Decision Log：agent 每个 turn 的 reasoning 摘要，便于事后审计
- Dashboard：per-agent 健康度、cost trend、tool 失败率

3.8.4 Cost Guard（三级熔断）

```
per-agent budget    → 单 agent 单次 / 单日预算
per-tenant budget   → 租户月预算
global circuit      → 全局成本告急熔断（防失控）
```

3.8.5 Multi-tenancy

- Namespace 隔离：`tenant_id` 贯穿 envelope / memory key / metric label
- 配额：per-tenant QPS / concurrency / cost
- 数据隔离：memory / audit log 物理或逻辑隔离

3.8.6 Secrets

- Vault / KMS 集中管理
- Agent 通过 short-lived token 拿 secret
- 审计：哪个 agent 拿了哪个 secret

---

4. 端到端案例：定时邮件回复 / 定期推送

4.1 任务拆解

普通人听到“自动回复邮件”会写脚本+cron 完事。Senior 要拆出来：

子能力	触发方式	形态
新邮件到达立刻判断要不要回	流式 (Gmail webhook)	reactive
起草回复内容	流式（接 triage 输出）	reactive
发送前要不要审批	流式 + human-in-loop	reactive
超过 24h 没回的提醒 / 重发	定时 (cron)	periodic
学习用户的回复风格	隐式背景	写 memory

这一拆 → 流式 + 定时 + multi-agent + memory + approval 全用上了。

4.2 Multi-Agent 设计

四个 agent 协作完成一件事，用户只写 manifest，不写一行平台代码：

```yaml
# ============ triage agent（流式入口）============
agent: email-triage
type: reactive
model: { primary: claude-haiku-4.5 }      # 分类用 Haiku 够
triggers:
  - event: email.received
tools: [gmail.read, contact.lookup]
memory:
  semantic: vector://user-email-style
output:
  emit_events: [email.draft.requested, email.ignored]
budget: { tokens_per_run: 5k, cost_cap_usd: 0.01 }

---
# ============ draft agent（起草回复）============
agent: email-draft
type: reactive
model: { primary: claude-sonnet-4.6, fallback: haiku }
triggers:
  - event: email.draft.requested
tools: [gmail.read, gmail.draft]
memory:
  semantic: vector://user-email-style
  episodic: pg://email-history
output:
  emit_events: [email.draft.ready]
budget: { tokens_per_run: 20k }

---
# ============ followup agent（定时巡检）============
agent: email-followup
type: periodic
model: { primary: haiku }
triggers:
  - cron: "*/30 * * * *"                  # 每 30 分钟
tools: [gmail.search]
output:
  emit_events: [email.draft.requested]    # 复用 draft agent

---
# ============ send agent（高危必审批）============
agent: email-send
type: reactive
triggers:
  - event: email.draft.ready
tools:
  - { id: gmail.send, require_approval: true }
output:
  emit_events: [email.sent, email.send.rejected]
```

4.3 端到端时序：一封邮件的旅程

```
[Gmail Push API]
      │
      ▼  ① WebhookEmitter 收到推送
┌──────────────────────────────────────────────────────────────┐
│ envelope: {                                                  │
│   topic: "email.received",                                   │
│   dedup_key: "msg:abc123",                                   │
│   payload: { msg_id, from, subj, .. }                        │
│ }                                                            │
└──────────────────────────────────────────────────────────────┘
          │
          ▼  ② Bus.publish
      Event Bus
          │
          ▼  ③ Router: email.received → email-triage
      Router
          │
          ▼  ④ Agent Runtime 拉起 triage agent
┌──────────────────────────────────────────────────────────────┐
│ triage:                                                      │
│   1. 拉 semantic memory (用户风格)                            │
│   2. tools.gmail.read(msg_id)                                │
│   3. tools.contact.lookup(from)                              │
│   4. LLM(haiku): 判断 important?                             │
│   5a. 不重要 → emit "email.ignored"                          │
│   5b. 重要   → emit "email.draft.                            │
│                      requested"                              │
└──────────────────────────────────────────────────────────────┘
          │
          ▼  ⑤ 新事件回 Bus
      Event Bus
          │
          ▼  ⑥ Router → email-draft
┌──────────────────────────────────────────────────────────────┐
│ draft:                                                       │
│   1. 拉 episodic (该联系人历史回复)                           │
│   2. 拉 semantic (用户风格 top-K)                             │
│   3. tools.gmail.read 拿全文+thread                           │
│   4. LLM(sonnet): 起草回复                                    │
│   5. tools.gmail.draft(content)                              │
│   6. emit "email.draft.ready"                                │
└──────────────────────────────────────────────────────────────┘
          │
          ▼  ⑦ Bus → Router → email-send
┌──────────────────────────────────────────────────────────────┐
│ send:                                                        │
│   1. tools.gmail.send(...)                                   │
│      ↓                                                       │
│      Tool Gateway 检测 require_                              │
│      approval=true → 暂停                                    │
│      ↓                                                       │
│      发审批到 Slack: "回复给 X 关于                           │
│      Y? [Approve] [Reject] [Edit]"                           │
│   2. 等用户回调                                               │
│   3. Approve → 真正发送                                       │
│      → emit "email.sent"                                     │
└──────────────────────────────────────────────────────────────┘
          │
          ▼  ⑧ 审计 + memory 写入
  - episodic: 写入 thread + 决策 + 结果
  - semantic: 若用户 Edit 后才发,
              把"用户编辑前后的 diff"
              作为新经验写入向量库
              (这就是隐式学习风格)
```

4.4 定时这条线（24h 未回提醒 / 定期推送）

```
[Cron: */30min]
       │
       ▼ CronEmitter（单实例 leader, timing wheel）
   Bus: cron.tick.email-followup
       │
       ▼ Router → email-followup
┌──────────────────────────────────────────────────────────────────────────┐
│ followup:                                │
│   1. tools.gmail.search(                 │
│        "to:me older_than:1d              │
│         -has:my_reply")                  │
│   2. 对每封 → emit                       │
│      "email.draft.requested"             │
│   3. 复用 draft agent 的链路！           │
└──────────────────────────────────────────────────────────────────────────┘
```

> 关键洞察：定时这条线没有重复实现 draft 逻辑，只是产生事件，复用流式管线。这就是事件总线架构的复用红利。

4.5 通用化：定期推送场景

把“邮件”换成“日报推送”，架构一行不动：

```yaml
agent: daily-digest
type: periodic
triggers:
  - cron: "0 9 * * *"                      # 每天 9 点
tools: [metric.query, news.fetch, slack.send]
memory:
  semantic: vector://user-interest
  episodic: pg://digest-history
budget: { tokens_per_run: 30k }
```

执行流程：

1. Cron tick → Bus → daily-digest agent
2. Agent 调 `metric.query` 拉昨天数据
3. Agent 调 `news.fetch` 拉相关行业新闻
4. Agent 拉 semantic memory（用户感兴趣的话题）
5. LLM 整合成摘要
6. Agent 调 `slack.send` 推送
7. 写入 episodic（用户点了哪几条 → 下次更精准）

4.6 关键设计细节

流 + 定时怎么“统一”

- 邮件到达：`email.received`（流式）
- 24h 未回：`cron.tick.email-followup` → `email.draft.requested`（定时）

- 两条路径下游汇合到同一个 draft agent，agent 不需要知道自己是被哪种触发器叫醒的

风格学习（memory 真正发挥价值）

- 用户每次 Approve 直接发 → 正样本
- 用户 Edit 后再发 → diff 是高价值学习信号
- 用户 Reject → 负样本
- 写入 semantic memory，下次 draft 时 retrieval-on-demand
- 这是 agent 框架对比纯脚本的核心优势：自迭代

高危操作（gmail.send）的处理

- Tool manifest 标记 `require_approval: true`
- Tool Gateway 拦截 → Approval Hook → Slack 卡片
- 用户响应前 agent 状态持久化（checkpoint）
- 审批回调通过事件总线送回（`approval.granted` / `approval.rejected`）
- 30 秒 Gmail undo 窗口期作为最后防线

误判保护

- Triage 误判“不重要” → 用户手动回了 → 监听 `email.user_replied`，写入 negative example
- Draft 质量差 → 用户 Reject → 同样进 memory
- 每个错误都成为下一次的训练数据

成本控制

步骤	模型	单封成本
Triage	Haiku	~$0.001
Draft	Sonnet	~$0.01
小计		~$0.011

100 封/天 ≈ $1.1/天。per-agent 日预算 cap，超了停掉 + 通知。

幂等

- `dedup_key = "msg:" + message_id`
- Gmail 偶尔重复推送 → Router 直接丢弃
- Send 操作用 `idempotency_key = "send:" + draft_id`，重试不会发两遍

4.7 错误场景演练

场景	处理
Gmail webhook 漏推	followup agent 兜底（cron 定期扫描）
Triage 把重要邮件判成垃圾	监听用户手动回复事件 → 写负样本
Draft agent 写出胡话	Output Schema 校验 + Approval 强制人审
审批超时未响应	24h 后 emit `approval.timeout` → 转入 followup
误发已发送	30s undo + 审计日志 + 告警
Anthropic API 挂了	Model Router fallback 到 OpenAI
Agent 崩溃	checkpoint 恢复 + episodic memory replay
死循环（auto-reply 互相回复）	trace_id 深度限制 + 同收件人冷却时间

4.8 通用化潜力

把“邮件”换成别的 task，架构一行不动，只换 manifest + tools：

Task	流式触发	定时触发	关键工具
自动回邮件	Gmail webhook	24h 未回扫描	`gmail.*`
Slack 自动响应	Slack event	提醒未读	`slack.*`
PR 自动 review	GitHub webhook	每天 stale PR 报告	`github.*`
Issue 自动 triage	issue 创建	周例会前总结	`linear/jira.*`
实验监控	metric 异常	小时巡检	`prometheus.*`
文献追踪	arXiv RSS	每周 digest	`arxiv.*`
客服自动应答	客户消息	SLA 检查	`intercom.*`
日报 / 周报推送	—	每天 / 每周 cron	`metric.* + slack.*`

> 这就是“通用 auto-task 平台”的真正含义：场景越多越证明抽象正确。

---

5. 关键设计决策与 Trade-offs

维度	选择	代价	理由
Bus 选型	Kafka	不原生支持 delay queue	吞吐 + 生态最好；delay 用 Pulsar 或外挂
一致性	at-least-once + 业务幂等	业务侧扛去重	exactly-once 代价远超收益
Cron 精度	分钟级走 bus；秒级走本地 timer	两套实现	总线延迟天然 ~100ms，秒级要绕开
Worker 模型	stateless 优先；长任务用 actor	两种 runtime	默认便宜，重场景才上成本
Agent vs Function	混合架构	双轨复杂	能用 function 别上 agent
Memory 隔离	默认 per-agent，shared 显式	协作要写代码	安全 + 简单优先
Tool 路径	业务工具走 Gateway，热路径白名单直连	治理粒度细	平衡延迟与审计
Model 路由	静态默认 + LLM 兜底	双轨	80% 场景静态够用
跨 agent 通信	走总线，不走 RPC	多一跳延迟	解耦 + 可观测 + 可重放

---

6. 演进路径

```
v0  cron + 脚本调 LLM            (一晚上能做完，但不可扩展)
       ↓
v1  加 webhook 支持流式          (流定时分离，两套代码)
       ↓
v2  抽出 Event Bus               (统一触发 → 第一次架构升华)
       ↓
v3  triage/draft/send 分 agent   (multi-agent → 复用 + 解耦)
       ↓
v4  加 Memory + 风格学习          (agent 真正"懂"用户)
       ↓
v5  接入通用平台                  (一份 manifest 跑邮件,
                                   再写一份跑 Slack / PR / ...)
       ↓
v6  多机房 + 多租户 + Marketplace  (平台化)
```

阶段	关键能力	估算工时
v0	单脚本 + cron	1 天
v1	+ webhook + 流式	3 天
v2	+ Event Bus	1 周
v3	+ multi-agent	2 周
v4	+ Memory 三层	2 周
v5	+ manifest 平台	1 月
v6	+ 多租户 / 多机房	季度

---

7. 总结

7.1 一句话架构

> 把 Trigger / Event / Agent 三层用事件总线串起来： Trigger 把异构源标准化进 Bus；Router 按事件分发到 Agent Runtime；Agent 通过 Memory / Tool / Model 三个 Gateway 完成 think-act-observe 循环；输出可能 emit 新事件回 Bus 形成 multi-agent 协作；Cross-cutting 提供状态、可观测、成本、安全能力贯穿全链路。

7.2 三性如何落地

目标	实现手段
效率	微批 + 分区 + 池化 + 模型分级 + Prompt Cache
可扩展性	事件总线解耦 + 声明式 manifest + Schema Registry + Topic 通配
通用性	Trigger / Event / Agent 三层正交，新场景 = 写 manifest

7.3 流式 + 定时统一的本质

```
Long-Running Agent:
    wait_any( inbox , next_wakeup )
              ↑           ↑
              流式        定时
              事件        唤醒
```

> 流式给 inbox，定时给 timer，Agent 自己决定下一次什么时候醒——这是流式与定时的真正统一。

7.4 多智能体协作的本质

不要 RPC，全部走事件总线： - Multi-agent = single-agent 的自然扩展 - 解耦：A 不需要知道 B 存在 - 可观测：所有协作链路在 trace 里 - 可重放：把事件重发就能重现整个协作

7.5 邮件案例的启示

“自动回复邮件”看似简单，但同时包含 流式触发、定时触发、multi-agent 协作、记忆学习、人机协作审批 五个维度——正好把架构每一层都用上。

> 架构的价值在于：今天解决邮件，明天换工具就能解决 Slack/PR/监控；今天 4 个 agent，明天扩到 40 个也是改 manifest 不改平台。

---

8. 工作预期

希望从事大模型后训练相关工作，在参与 SFT、RLHF 及模型对齐优化的基础上，重点探索 agentic 能力在 AI persona 和社交场景中的落地，例如多轮对话中的目标驱动、长期记忆与人设一致性，以及在复杂互动中的策略决策能力。同时希望参与面向社交场景的数据构建与自动化 pipeline 设计，通过构造多轮交互与任务链数据，提升模型在真实沟通环境中的理解能力与响应质量。在实际工作中，通过 bad case 分析持续优化模型在用户互动中的表现，减少幻觉与不一致问题。长期来看，希望在大模型后训练与 agentic 社交智能方向持续深入，参与构建具备稳定人格与高质量互动能力的 AI 系统。同时也希望能融入积极向上的团队，在良性合作的氛围中共同成长。

文档版本：v1.0 | 设计目标：兼顾效率、可扩展性、通用性的多智能体 Auto-Task 架构